the available hardware, whether CPU or GPU, while using an appropriate PyTorch data type for efficient inference. Although the compressed model occupies only a few gigabytes during deployment, downloading the complete model requires approximately 10GB of storage.

Once the model is loaded, uploaded images are converted into tensor objects before being passed to the captioning model. The model analyses the image and generates a descriptive caption, which serves as the input for the next stage of the pipeline.

Generating image summaries with Gemma 2B

The summarisation is performed using the Gemma 2B language model. The Hugging Face text generation pipeline is built and configured with a repetition penalty.

The model occupies approximately 4.9GB of space and is used along with the prompt templates and LangChain's LLMChain to generate the summaries in a structured manner from the captions generated by PaliGemma. After the generation of summaries, the output is cleaned to deliver the result.

Integrating LangChain with Gemma

LangChain organises model capabilities into reusable tools that agents can invoke whenever required. In this implementation, the first tool performs image captioning by converting uploaded images into RGB format before passing them to the PaliGemma model for caption generation.

A second tool is responsible for text summarisation. Rather than performing an analysis on the images themselves, it uses the generated caption as its input and generates a short textual summary. Separating these functions into individual tools makes it possible for the LangChain agent to coordinate several AI models within one pipeline.

The LangChain agent acts as the central orchestrator of the application. It is initialised with the language model alongside two tools—the image captioning tool and the summarisation tool.

When an image is uploaded, the agent first invokes the captioning model to generate a textual description. It then forwards this caption to the summarisation model, cleans the generated output and returns a structured response containing both the detailed caption and its summary. The modular architecture allows combining various models in one workflow.

Creating the user interface with Gradio

Gradio offers the front end for the app. Users upload an image through the web interface, after which the LangChain pipeline processes the request and returns both the generated caption and summary.

Despite combining multiple AI models, the complete interface can be implemented using fewer than one hundred lines of code. The same implementation demonstrated in Google Colab can also be executed locally in development environments such as Visual Studio Code, where it operates successfully on systems equipped with approximately 16GB RAM.

Hardware optimisation and LangSmith

Unlike several higher-level orchestration frameworks, LangChain provides developers with greater control over hardware resources. It supports lower-level optimisation techniques that improve CPU, GPU and memory utilisation, making it particularly suitable for edge AI deployments.

For monitoring and debugging, LangSmith offers observability of the whole application. Workflow execution, reasoning processes, and agent interaction can be observed to facilitate problem solving and enhance transparency during development of the application using AI.

Choosing the right open source models

Different open source models are designed for different tasks. In this demonstration, Gemma 2B is selected because it performs well for text summarisation, while PaliGemma is better suited to image captioning.

Developers can experiment with different combinations of models to build applications that perform reverse workflows, such as generating images from summaries or validating whether uploaded images match accompanying documents. Understanding which model is best suited for a particular task remains an important part of developing efficient AI applications. END

Balancing performance and resource constraints

Running AI models locally comes with some trade-offs. Although edge computing guarantees that users' privacy is protected and applications run without internet connectivity, inference for these takes longer when compared to the cloud.

Quantization is an optimization method. For instance, converting a 10GB model to 4-bit quantization can reduce its size to approximately 4GB. In addition, model distillation and lightweight architectures make it possible to implement AI on limited resources efficiently. Developers should implement guardrails like pre- and post-processing hooks to increase safety and effectiveness in edge AI.

This article is based on the talk 'Using LangChain and Gemma models to develop faster AI agents on edge' by Supriya Sirbi, Software Architect, Aptiv, at AI DevCon India 2026. It has been transcribed by Ananthu Ashok, journalist, EFY Group.